DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

(Artificial Intelligence & Machine Learning)

Academic Year:	2026-27	**Name of Student:**	
Semester:	V	**Student ID:**	
Class / Branch:	TE – CSE (AI&ML)	**Roll No:**	
Subject:	WMD Lab	**Date of Submission:**	

Experiment No. 08

Lab Outcome: Apply microservices, REST APIs, and deployment techniques for scalable applications.

Aim: Implement user authentication using JWT/OAuth2 and test with Postman.

Software used: Python, Flask, PyJWT,Postman

Theory: -

What is an API and a REST API?

An **API (Application Programming Interface)** allows two software applications to communicate with each other. It works like a waiter in a restaurant: the customer (client) never enters the kitchen (server); the waiter (API) carries the request to the kitchen and brings back the result.

REST (Representational State Transfer) is a set of principles for communicating with resources over HTTP. Each URL, such as /register, /login and /dashboard, represents an API endpoint.

Common HTTP Methods

HTTP Method	Purpose	Example
GET	Retrieve data	/dashboard
POST	Create / send data	/register
PUT	Update a complete resource	/student/101
PATCH	Partially update a resource	/student/101
DELETE	Delete a resource	/student/101

In this experiment we use POST /register, POST /login and GET /dashboard.

HTTP Statelessness

Each HTTP request is treated independently. When a student sends GET /dashboard, the server must determine which user is making the request. A JWT supplies this identity information with every request, so the server does not need to remember the user between requests.

Authentication vs Authorization

Term	Question answered	Example
Authentication	Who are you? (identity verification)	Username + password, followed by a successful login
Authorization	What are you allowed to access? (permission verification)	A valid JWT permits access to /dashboard

Password Hashing

Passwords must never be stored directly in a database. If the database is compromised, plain-text passwords would be exposed. Instead, the password is passed through a hashing algorithm and only the resulting hash is stored:

```
Original Password  -->  Hashing Algorithm  -->  Hashed Password

mypassword123  -->  scrypt:32768:8:1$........
```

Hashing vs Encryption: Hashing is designed to be one-way (a hash cannot be reversed to get the password), whereas encryption can normally be reversed with the appropriate key. Password storage therefore uses hashing, not reversible encryption.

Flask applications use the werkzeug.security module, which provides two important functions: generate_password_hash() to create a hash, and check_password_hash() to verify a supplied password against a stored hash. The original password does not need to be stored.

JSON Web Token (JWT)

JWT stands for **JSON Web Token**. It is a compact token format commonly used for authentication and authorization. After a successful login, the server issues a signed token; the client then sends this token when accessing protected resources.

A JWT has three parts separated by dots: **HEADER.PAYLOAD.SIGNATURE**

Part	Contents	Example
Header	Token type and signing algorithm	{"alg": "HS256", "typ": "JWT"}

Part	Contents	Example
Payload	Claims such as username and expiry (sub, iat, exp, iss, aud)	{"username": "vijesh", "exp": 1790000000}
Signature	Generated from Header + Payload + Secret Key; detects tampering	Computed with HMAC-SHA256

Is a JWT encrypted? A normal signed JWT is **not** encrypted. Its payload can be decoded by anyone who holds the token. The signature provides integrity and authenticity, not confidentiality. Therefore, passwords, credit-card numbers or other confidential data must never be placed in a normal JWT payload.

PyJWT Library

PyJWT is a Python library that implements the JWT standard (RFC 7519). It is used to create tokens, decode tokens, verify signatures and check expiration. It is installed with: pip install PyJWT. Its two core operations are shown below.

jwt.encode() packs user data and an expiration time into a payload, signs it with a secret key and returns the token string:

```
import jwt, datetime

payload = {
    'user_id': 101,
    'exp': datetime.datetime.now(datetime.timezone.utc)
           + datetime.timedelta(hours=1)
}
secret_key = 'supersecretkey'

token = jwt.encode(payload, secret_key, algorithm='HS256')
```

jwt.decode() verifies that the token has not been tampered with or expired, and returns the original data. An expired token raises jwt.ExpiredSignatureError and any other invalid token raises jwt.InvalidTokenError:

```
try:
    decoded_data = jwt.decode(token, secret_key, algorithms=['HS256'])
    print(decoded_data['user_id'])      # Outputs: 101
except jwt.ExpiredSignatureError:
    print("Token has expired!")
except jwt.InvalidTokenError:
    print("Invalid token signature!")
```

PyJWT supports symmetric algorithms (HMAC with SHA-256, i.e. HS256) as well as asymmetric

algorithms (RSA and Elliptic Curve key pairs).

JWT Authentication Flow

```
USER --> POST /login (username + password)
            |
            v
     Flask verifies password hash
            |
     Password correct?
       No  --> 401 Unauthorized
       Yes --> Create JWT --> Return JWT to client

CLIENT --> GET /dashboard  (Authorization: Bearer <JWT>)
            |
            v
     Flask verifies JWT (signature + expiry)
       Invalid --> 401 Unauthorized
       Valid   --> Dashboard data (200 OK)
```

Bearer Token and Middleware

A JWT is normally sent in the HTTP Authorization header in the format **Authorization: Bearer <JWT_TOKEN>**. The word Bearer indicates that the client is presenting a token as its credential.

Middleware is logic that sits between the incoming request and the final endpoint. In this program, the /dashboard endpoint itself checks the Authorization header and validates the JWT before returning data. In larger applications, this verification is moved into reusable middleware or decorators.

JWT vs Session-Based Authentication

	Traditional Session	JWT
Process	Server creates a session after login and stores it; client receives a session identifier	Server creates a signed JWT; client stores it and sends it with each request
State	Server maintains session state	Server verifies the token; less need for server-side session storage

JWT vs OAuth 2.0

JWT and OAuth 2.0 are not the same thing. **JWT is a token format**, whereas **OAuth 2.0 is an authorization framework** that defines how a client obtains delegated access to protected resources on behalf of a resource owner (through an authorization server and a resource server).

OAuth 2.0 systems may use JWTs as access tokens, but JWT can also be used independently.

In this experiment, the appropriate description is: **JWT-based authentication using Flask and PyJWT**. The same Bearer-token mechanism is what an OAuth 2.0 resource server uses to validate access tokens.

HTTP Status Codes Used

Code	Meaning	Used when
200	OK	Successful login or dashboard access
201	Created	Successful registration
400	Bad Request	Missing fields or duplicate username
401	Unauthorized	Wrong password, or missing / invalid / expired JWT

Security principles demonstrated

- **Password hashing:** plain password → hash → storage.
- **Token authentication:** login → JWT → protected resource.
- **Signature verification:** a modified token fails verification, so an attacker cannot change "vijesh" to "admin" without the secret key.
- **Token expiration:** the exp claim limits how long a stolen token remains usable.
- **Authorization header:** tokens are sent as Authorization: Bearer <token>.

Limitations and production improvements

This program is a demonstration only. Because users_db is stored in RAM, all users disappear when the server stops. A real admission portal should additionally use:

- **A persistent database** (PostgreSQL, MySQL, SQLite or MongoDB).
- **Secret management:** load SECRET_KEY from an environment variable, e.g. os.environ.get("SECRET_KEY"), instead of hard-coding it.
- **HTTPS** to protect credentials and tokens in transit.
- **Input validation** of username format, password strength, request structure and data types.
- **Rate limiting** on the login endpoint against automated attempts.
- **Careful token design:** token lifetime, claims, key management, refresh tokens and revocation policies.

Key Features Implemented

- **Password Hashing:** passwords are never stored in plain text; they are hashed using werkzeug.security.
- **Stateless Token Authentication:** signed JWT tokens with an expiration time (exp) manage user sessions without server-side session overhead.
- **Token Verification:** private endpoints are protected by validating the Bearer token in the Authorization header.
- **Error Handling:** clear JSON error responses for missing fields, incorrect credentials, expired tokens and missing headers.

API Endpoints

Endpoint	Method	Purpose	Authentication
/register	POST	Register a new user	Not required
/login	POST	Authenticate user and issue JWT	Not required
/dashboard	GET	Access protected resource	JWT required

Problem Statement

Educational institutions require secure web portals where students can register, log in, and access protected resources such as admission status, event registrations, examination and attendance information, and personal profile dashboards.

To implement user authentication using JWT/OAuth2 concepts in a Python Flask REST API and to test the Registration, Login and Protected Dashboard endpoints using Postman.

The API developed in this experiment:

- Allows users to register.
- Stores passwords securely using password hashing.
- Authenticates users during login.
- Generates a JSON Web Token (JWT) after successful authentication.
- Protects restricted resources using JWT authentication.
- Handles authentication errors appropriately.

HTTP is fundamentally **stateless**: the server does not automatically remember a user from one request to the next. After a student logs in (Request 1), the server needs a secure way to identify that same student when the dashboard (Request 2) or admission status (Request 3) is requested.

One solution is **token-based authentication**, which is implemented in this experiment using Python, Flask, PyJWT and JWT tokens.

Software Used

Software / Library	Purpose
Python 3	Programming language used to write the API
Flask	Lightweight web framework used to build the REST API
PyJWT	Creates and verifies JSON Web Tokens
Werkzeug (installed with Flask)	Provides password hashing utilities
Postman	API testing tool used to send requests and inspect responses

Prerequisites and Project Setup

Verify that Python is installed using python --version, then install the required packages. Werkzeug is installed automatically as a Flask dependency.

```
pip install Flask PyJWT
```

Create the project folder D:\FlaskJWT and, inside it, a single file named app.py. All the code sections in the next part are placed together, in order, in this file.

Program Implementation

The program is explained section by section. Each section first states the theory, then what the code does, why it is important and what comes next, followed by the code. Together, the sections form the complete app.py.

Part A: Imports, Application Setup and User Store

Theory: Every Flask API begins by importing the required libraries and creating the application object. The secret key is the value used to sign every JWT; anyone who knows it can forge valid tokens. Registered users are kept in a Python dictionary in the form { username: hashed_password }, which acts as a simple in-memory database for demonstration.

What the code does: imports Flask, request (reads client data) and jsonify (creates JSON responses); imports PyJWT, datetime (for token expiry) and werkzeug.security (for hashing); creates the Flask app; defines the secret key; and creates the empty users_db dictionary.

Why it is important: this section provides the foundation on which all three endpoints depend.

The secret key underpins the security of the token system. In production it must be read from an environment variable, and the dictionary must be replaced by a persistent database such as MySQL, PostgreSQL, SQLite or MongoDB, since data in RAM is lost when the server stops.

Next: the /register endpoint uses users_db and the hashing functions to create new users.

```
from flask import Flask, request, jsonify
import jwt
import datetime
import werkzeug.security as security

app = Flask(__name__)

# Secret key used to sign and verify JWT tokens
app.config['SECRET_KEY'] = 'admission_portal_secret_key_123'

# In-memory database: { "username": "hashed_password" }
users_db = {}
```

Part B: User Registration Endpoint (/register)

Theory: Registration creates a new resource (a user account), so it uses the POST method and returns HTTP status 201 (Created) on success. Input must always be validated before it is used: a request with missing fields or an already existing username is rejected with status 400 (Bad Request). The most important step is hashing the password before storage.

What the code does: reads the JSON body with request.get_json(); checks that both username and password are present; checks whether the username already exists in users_db; stores generate_password_hash(password) against the username; and returns a success message with status 201.

Why it is important: this is where the security principle of never storing plain-text passwords is enforced. Even if users_db is leaked, attackers obtain only hashes and not the actual passwords. Input validation also prevents duplicate accounts and malformed requests.

Next: once a user is registered, the /login endpoint verifies the password against the stored hash and issues a JWT.

```
@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()

    if not data or 'username' not in data or 'password' not in data:
        return jsonify({'error': 'Username and password are required'}), 400
```

```
    username = data['username']
    password = data['password']

    if username in users_db:
        return jsonify({'error': 'User already exists!'}), 400

    # Hash the password for security
    users_db[username] = security.generate_password_hash(password)
    return jsonify({'message': f'User {username} registered successfully!'}), 201
```

Part C: User Login Endpoint (/login) – JWT Generation

Theory: Login is the authentication step ("Who are you?"). The server retrieves the stored hash, verifies the supplied password with check_password_hash(), and, if correct, creates a JWT. The payload contains the username and an exp (expiration) claim set to 30 minutes from the current time. The payload and secret key are passed to jwt.encode() with the HS256 algorithm. A wrong username or password returns 401 (Unauthorized), using the same message for both cases so that an attacker cannot tell which one was wrong.

What the code does: validates the request fields; looks up the stored hash with users_db.get(username); verifies the password against it; builds the payload with username and expiry; signs it into a token; and returns the token with status 200.

Why it is important: this is the core of the experiment. It converts a one-time password check into a signed, time-limited token, so that the client does not need to send the password again with every request. The expiration claim limits the damage if a token is stolen.

Next: the client keeps the token and sends it in the Authorization header to the protected /dashboard endpoint.

```
@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()

    if not data or 'username' not in data or 'password' not in data:
        return jsonify({'error': 'Username and password are required'}), 400

    username = data['username']
    password = data['password']

    hashed_password = users_db.get(username)

    # Verify user existence and password hash match
    if not hashed_password or not security.check_password_hash(
            hashed_password, password):
        return jsonify({'error': 'Invalid username or password'}), 401
```

```
    # Create JWT payload with expiration time (30 minutes)
    payload = {
        'username': username,
        'exp': datetime.datetime.now(datetime.timezone.utc)
               + datetime.timedelta(minutes=30)
    }

    # Generate token
    token = jwt.encode(payload, app.config['SECRET_KEY'], algorithm='HS256')

    return jsonify({
        'message': 'Login successful!',
        'token': token
    }), 200
```

Part D: Protected Dashboard Endpoint (/dashboard)

Theory: This endpoint demonstrates authorization ("What are you allowed to access?"). The client must send the header Authorization: Bearer <token>. The server extracts the token, then calls jwt.decode() with the secret key and the expected algorithm. PyJWT verifies the token structure, the signature, the algorithm and the expiration time. If an attacker changes the payload (for example from "vijesh" to "admin"), the signature no longer matches and verification fails.

What the code does: reads the Authorization header; rejects the request with 401 if the header is missing or does not start with "Bearer "; extracts the token; decodes and verifies it; returns the dashboard data for the user named in the payload; and handles jwt.ExpiredSignatureError and jwt.InvalidTokenError with clear 401 messages.

Why it is important: this is the section that actually protects a resource. It proves that only a client holding a valid, unexpired, untampered token can obtain private data, and it shows how authentication errors are reported to the client.

Next: the final section starts the Flask server so that all endpoints can be tested in Postman.

```
@app.route('/dashboard', methods=['GET'])
def dashboard():
    auth_header = request.headers.get('Authorization')

    if not auth_header or not auth_header.startswith('Bearer '):
        return jsonify({'error': 'Authorization header missing or improperly
formatted'}), 401

    # Extract token string after 'Bearer '
    token = auth_header.split(' ', 1)[1]
```

```
    try:
        # Decode and verify the token
        decoded_payload = jwt.decode(token, app.config['SECRET_KEY'],
algorithms=['HS256'])
        username = decoded_payload['username']

        return jsonify({
            'message': f'Welcome to your Admission Dashboard, {username}!',
            'admission_status': 'Application Under Review',
            'events_registered': ['AIML Club Hackathon 2026']
        }), 200

    except jwt.ExpiredSignatureError:
        return jsonify({'error': 'Token has expired. Please log in again.'}), 401
    except jwt.InvalidTokenError:
        return jsonify({'error': 'Invalid token. Authorization denied.'}), 401
```

Part E: Running the Flask Server

Theory: Flask includes a built-in development server. The block below runs only when the file is executed directly (python app.py) and not when it is imported as a module.

What the code does: starts the server on port 5000 with debug mode on, so the API is available at http://127.0.0.1:5000 and the server reloads automatically when the code changes.

Why it is important: without this section the endpoints exist but cannot receive requests. Debug mode is convenient for learning but must be turned off in production.

Next: run the program and test each endpoint using Postman.

```
# Run the server
if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

Note: the token expiry uses datetime.datetime.now(datetime.timezone.utc). The older datetime.utcnow() is deprecated in recent Python versions.

D: > AY_2026-27 > ODD > WMD_VJshNair > Lab > Exp-8 > Program > app.py

```python
from flask import Flask, request, jsonify
import jwt
import datetime
import werkzeug.security as security

app = Flask(__name__)

# Secret key used to sign and verify JWT tokens
app.config['SECRET_KEY'] = 'admission_portal_secret_key_123'

# In-memory database storing registered users: { "username": "hashed_password" }
users_db = {}

# ------------------------------------------------------------
# 1. USER REGISTRATION ENDPOINT
# ------------------------------------------------------------
@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()

    if not data or 'username' not in data or 'password' not in data:
        return jsonify({'error': 'Username and password are required'}), 400

    username = data['username']
    password = data['password']

    if username in users_db:
        return jsonify({'error': 'User already exists!'}), 400

    # Hash the password for security
    users_db[username] = security.generate_password_hash(password)
    return jsonify({'message': f'User {username} registered successfully!'}), 201

# ------------------------------------------------------------
# 2. USER LOGIN ENDPOINT (Generates JWT Token)
# ------------------------------------------------------------
@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()

    if not data or 'username' not in data or 'password' not in data:
        return jsonify({'error': 'Username and password are required'}), 400

    username = data['username']
    password = data['password']

    hashed_password = users_db.get(username)

    # Verify user existence and password hash match
    if not hashed_password or not security.check_password_hash(hashed_password, password):
        return jsonify({'error': 'Invalid username or password'}), 401

    # Create JWT payload with expiration time (30 minutes)
    payload = {
```

```python
        'username': username,
        'exp': datetime.datetime.utcnow() + datetime.timedelta(minutes=30)
    }

    # Generate token
    token = jwt.encode(payload, app.config['SECRET_KEY'], algorithm='HS256')

    return jsonify({
        'message': 'Login successful!',
        'token': token
    }), 200

# ------------------------------------------------------------
# 3. PROTECTED DASHBOARD ENDPOINT (Requires Valid JWT)
# ------------------------------------------------------------
@app.route('/dashboard', methods=['GET'])
def dashboard():
    auth_header = request.headers.get('Authorization')

    if not auth_header or not auth_header.startswith('Bearer '):
        return jsonify({'error': 'Authorization header missing or improperly formatted'}), 401

    # Extract token string after 'Bearer '
    token = auth_header.split(' ')[1]

    try:
        # Decode and verify the token
        decoded_payload = jwt.decode(token, app.config['SECRET_KEY'], algorithms=['HS256'])
        username = decoded_payload['username']

        return jsonify({
            'message': f'Welcome to your Admission Dashboard, {username}!',
            'admission_status': 'Application Under Review',
            'events_registered': ['AIML Club Hackathon 2026']
        }), 200

    except jwt.ExpiredSignatureError:
        return jsonify({'error': 'Token has expired. Please log in again.'}), 401
    except jwt.InvalidTokenError:
        return jsonify({'error': 'Invalid token. Authorization denied.'}), 401

# Run the server
if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

OUTPUT AND TESTING USING POSTMAN

Postman is an API testing tool that allows us to send HTTP requests, add headers, send JSON, inspect responses and test authentication and error conditions.

Step 1: Start the Application

Open Command Prompt or PowerShell, move to the project folder and run the script. Keep this terminal running throughout the testing.

```
cd D:\FlaskJWT
python app.py

 * Running on http://127.0.0.1:5000
```

```
PS D:\AY_2026-27\ODD\WMD_VJshNair\Lab\Exp-8\Program> python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 676-751-294
```

Fig. 1: Flask server started

Step 2: Register a New User

1. Open Postman and create a new request.
2. Set the Method to POST and the URL to http://127.0.0.1:5000/register.
3. Go to Body, select raw, and choose JSON from the dropdown.
4. Enter the JSON body below and click Send.

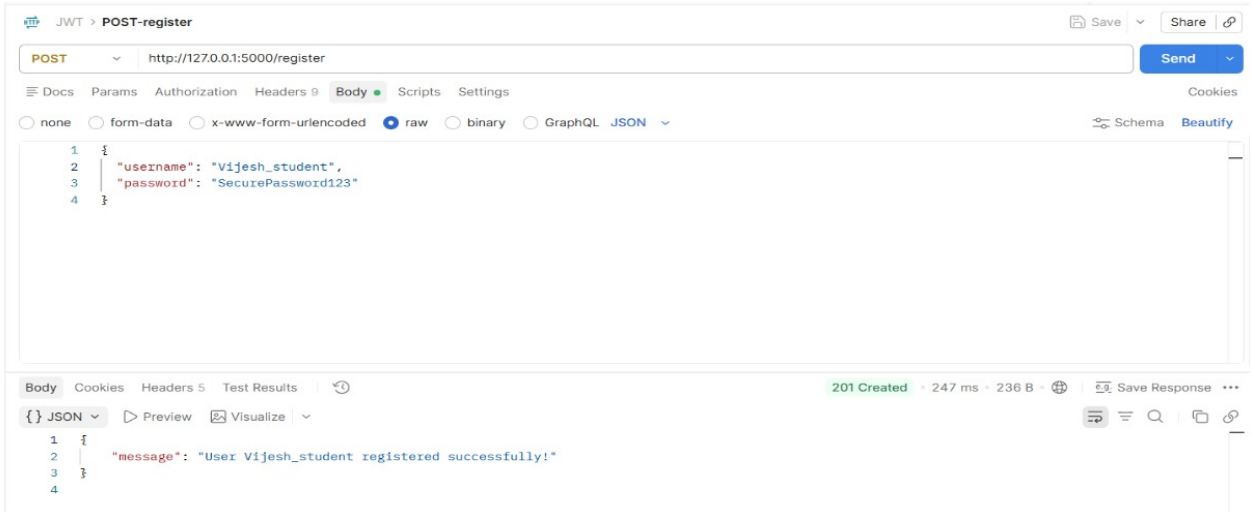


Fig. 2: User registration

Step 3: Login to Obtain the JWT Token

1. Open a new request tab. Set the Method to POST and the URL to http://127.0.0.1:5000/login.
2. Go to Body, then raw, then JSON, and enter the same username and password.
3. Click Send and copy only the token value from the response (not the "token": part).

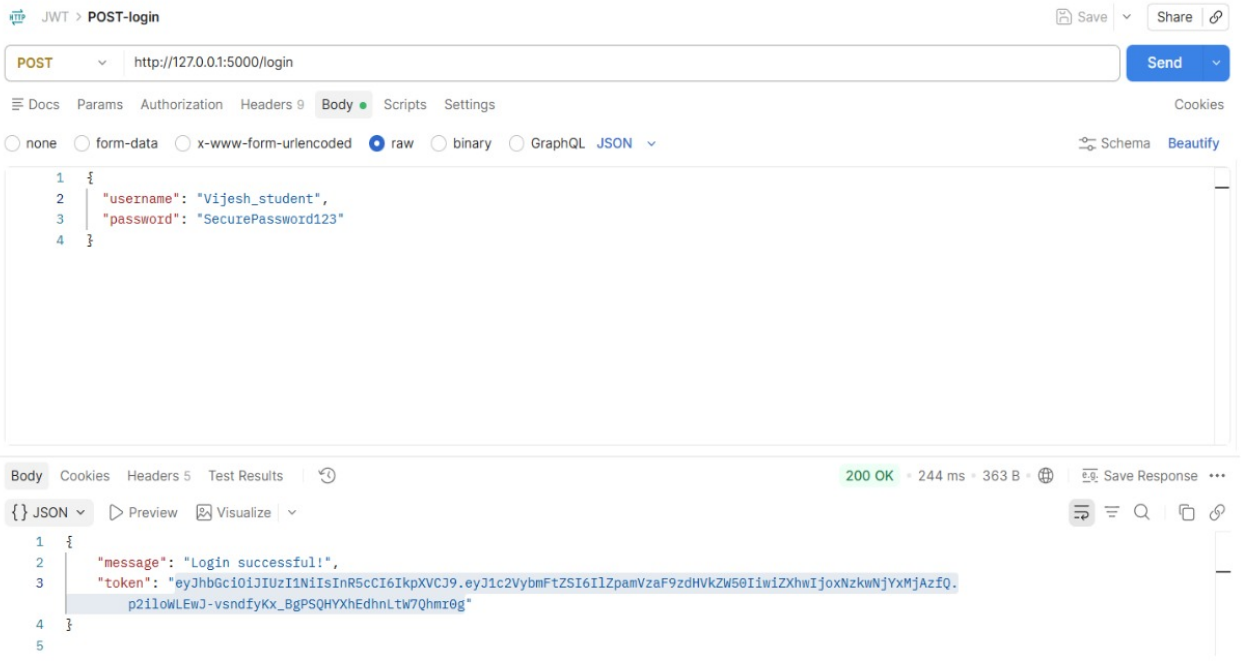


Fig. 3: Login and JWT generation

Step 4: Access the Protected Dashboard

1. Open a new request tab. Set the Method to GET and the URL to http://127.0.0.1:5000/dashboard.
2. Go to the Authorization tab and select Type: Bearer Token.
3. Paste the token copied in Step 3 into the Token field and click Send.

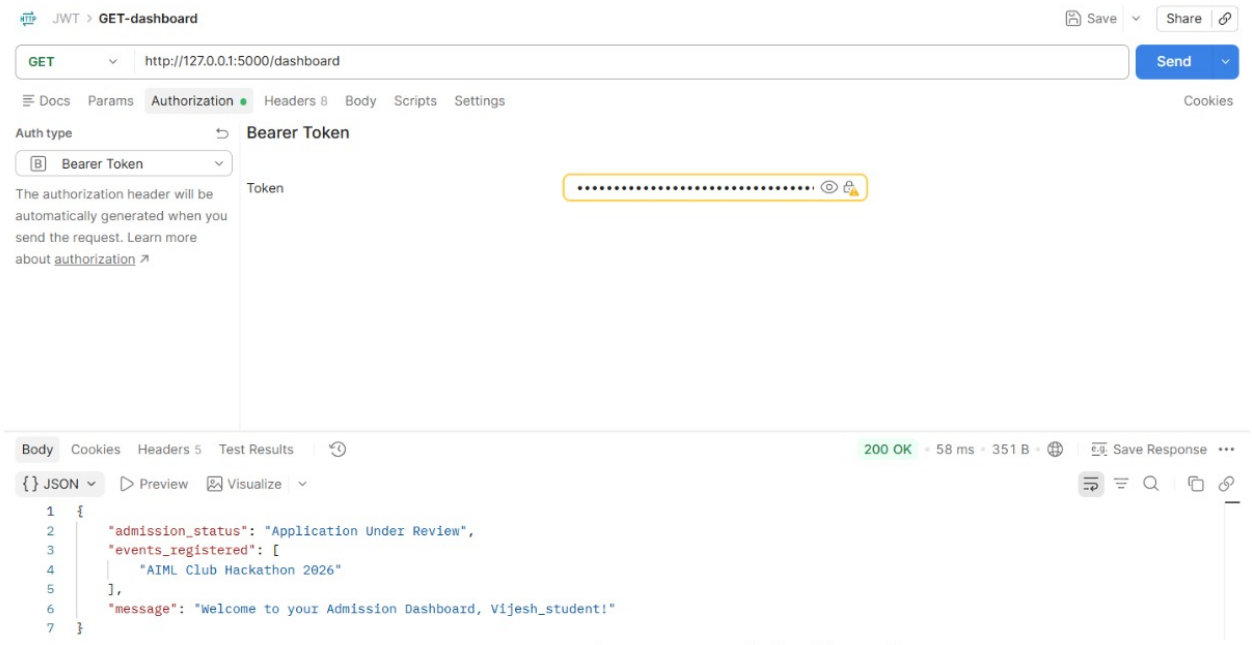


Fig. 4: Access to the protected dashboard

Step 5: Test Authorization Failure

In the Dashboard request, modify one or more characters of the token (or clear it) in the Authorization tab and click Send. The signature check fails and the server denies access.

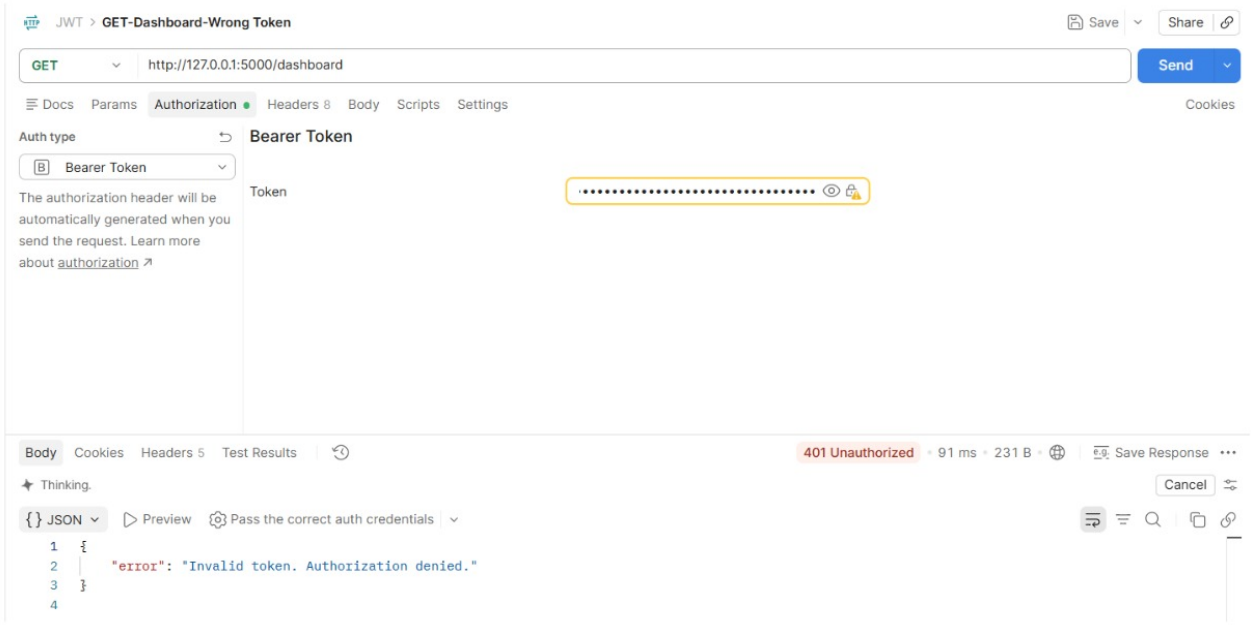
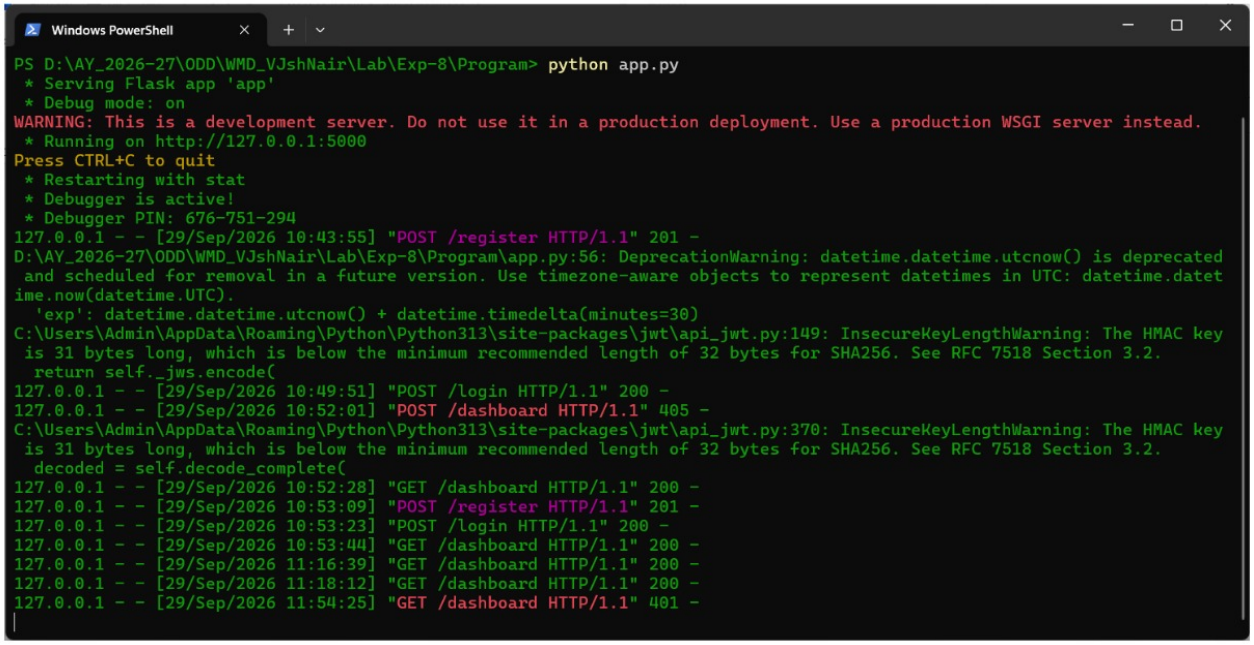


Fig. 5: Authorization failure



Conclusion: -

Thus, we have demonstrated and implemented user authentication using JWT/OAuth2 and test with Postman.